

Quais problemas de qualidade de dados podem ser identificados durante uma análise utilizando Python?
Thoughts

Problemas de Qualidade de Dados Identificáveis com Python e Pandas

Antes de produzir qualquer indicador, gráfico ou relatório automatizado, é fundamental diagnosticar e tratar problemas de qualidade presentes na base de dados. As fontes apontam quatro problemas principais que costumam afetar análises tabulares:

Tipos de Dados Incorretos (dtypes): Números armazenados como texto ou com símbolos e caracteres inválidos, o que impede a execução de cálculos matemáticos e estatísticos.

Valores Ausentes / Nulos (Missing Data): Células sem preenchimento ou dados faltantes que distorcem médias e somatórios.

Duplicidades (Duplicate Data): Linhas ou rótulos repetidos gerados por falhas de exportação de sistemas, inflacionando a contagem de registros.

Inconsistências de Preenchimento e Informações Fora do Padrão: Erros de digitação, variações de caixa (maiúsculas/minúsculas) e espaços em branco extras em campos de texto.

Como Identificar e Tratar Cada Problema Passo a Passo

1. Identificação e Correção de Tipos Incorretos

O Problema: Em planilhas, é comum encontrar colunas numéricas contendo textos (ex: "N/A", "Pendente") ou formatações que fazem o Pandas ler a coluna inteira como texto (object).

Identificação: Verifica-se a estrutura do DataFrame via atributo de tipos de colunas (dtypes).

Tratamento: Aplica-se a função `pd.to_numeric()` com o parâmetro `errors="coerce"`. Essa instrução força a conversão numérica dos valores válidos e transforma automaticamente qualquer entrada inválida ou textual no indicador nulo NaN.

```
# Converte a coluna 'Valor' para o tipo numérico.  
# Entradas em texto ou corrompidas viram NaN automaticamente.  
dados["Valor"] = pd.to_numeric(dados["Valor"], errors="coerce")
```

2. Identificação e Tratamento de Valores Ausentes

O Problema: Células vazias em colunas essenciais para o negócio (como valores ou status da transação).

Tratamento: Dependendo da regra de negócio, podem-se eliminar as linhas contendo nulos em colunas críticas com `.dropna()` ou preencher as lacunas com valores padrão com `.fillna()`.

```
# Elimina apenas os registros onde a coluna 'Valor' ficou nula/ausente  
dados_limpos = dados.dropna(subset=["Valor"])
```

3. Remoção de Duplicidades

O Problema: Linhas exatamente iguais no arquivo de entrada que duplicam lançamentos.

Tratamento: O Pandas possui o método nativo `.drop_duplicates()` para identificar e descartar repetições idênticas na tabela.

```
# Descarta linhas com registros completamente duplicados  
dados_unicos = dados_limpos.drop_duplicates()
```

4. Padronização de Textos e Remoção de Espaços Extras

O Problema: Inconsistências como "Pendente ", "pendente" e "PENDENTE", que o Pandas trataria como três categorias distintas.

Tratamento: Utilizam-se os métodos de string vetoriais do Pandas (`.str`) para remover espaços em branco nas extremidades (`.strip()`) e padronizar o formato do texto (`.capitalize()` ou `.lower()`).

```
# Remove espaços nas pontas e padroniza a caixa do texto
dados_unicos["Status"] = (
    dados_unicos["Status"]
    .astype(str)
    .str.strip()
    .str.capitalize()
)
```

Exemplo Prático Integrado de Limpeza

Abaixo apresenta-se o fluxo completo em Python para carregar uma planilha, diagnosticar e tratar os quatro problemas descritos e estruturar um DataFrame higienizado:

```
import pandas as pd

# 1. Carregamento do arquivo Excel bruto
dados = pd.read_excel("pagamentos.xlsx")

# 2. Correção de tipo de dado (valores não numéricos viram NaN)
dados["Valor"] = pd.to_numeric(dados["Valor"], errors="coerce")

# 3. Eliminação de registros nulos em colunas essenciais
dados_validos = dados.dropna(subset=["Valor", "Status"])

# 4. Remoção de linhas duplicadas
dados_unicos = dados_validos.drop_duplicates()

# 5. Padronização de textos (remoção de espaços e ajuste de caixa)
dados_unicos["Status"] = (
    dados_unicos["Status"]
    .astype(str)
    .str.strip()
    .str.capitalize()
)
```

```
# 6. Exibição da base tratada e pronta para análises/relatórios
print("Base higienizada com sucesso:")
print(dados_unicos)
```

Explicação passo a passo do código:

`pd.read_excel("pagamentos.xlsx")`: Abre o arquivo e constrói o DataFrame inicial na memória.

`pd.to_numeric(..., errors="coerce")`: Força a conversão da coluna "Valor" para número e converte erros de digitação em NaN.

`dados.dropna(subset=["Valor", "Status"])`: Garante que o relatório final não contenha dados incompletos nas colunas principais.

`dados_validos.drop_duplicates()`: Limpa a base removendo cópias idênticas.

`.str.strip().str.capitalize()`: Higieniza a coluna de texto, convertendo entradas como " pendente " em "Pendente".

Boas Práticas de Qualidade de Dados

Documentação do Processo: A limpeza e as regras de transformação aplicadas devem ser documentadas no próprio código para garantir a auditabilidade e reprodutibilidade do fluxo.

Validação com Dados de Teste: Teste o script de higienização com amostras pequenas antes de colocá-lo para rodar em produção de forma agendada.

Nomes Claros de Variáveis: Mantenha variáveis intermediárias bem nomeadas (`dados_validos`, `dados_unicos`) para rastrear o impacto de cada etapa.

Fontes que Sustentam esta Resposta

fonte_python_analise_automacao_dados.pdf: Seção 6 (Qualidade dos dados), Seção 7 (Exemplo integrado) e Seção 8 (Boas práticas).

User Guide — pandas 3.0.6 documentation: Módulos de Working with missing data, Duplicate Labels e Working with text data.

Getting started — pandas 3.0.6 documentation: Tópicos de manipulação de dados tabulares e limpeza de textos.